Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO




## EVALUACIÓN N° 03

# Desarrollo de aplicaciones Web: Microservicios.

DOCENTE:

Coello Palomino, Ricardo

CURSO:

Desarrollo de aplicaciones Web
4 - C24 - Sección D

# LABORATORIO 13: Gestión de proveedores y compras

**Estudiante:** León Fernández, Renzo Raúl **Sección:** D

**Situación problemática.** Una empresa necesita controlar proveedores, productos y compras.

**Objetivo.** Crear Proveedor, Producto y Compra con detalles.

## Requerimientos funcionales

### 1. CRUD de proveedores y productos.

Cada uno tiene su propio microservicio, y en ambos se puede registrar, listar, buscar por id, actualizar y eliminar. Además, al registrar o actualizar un producto, el servicio le pregunta al de proveedores si ese proveedor existe.

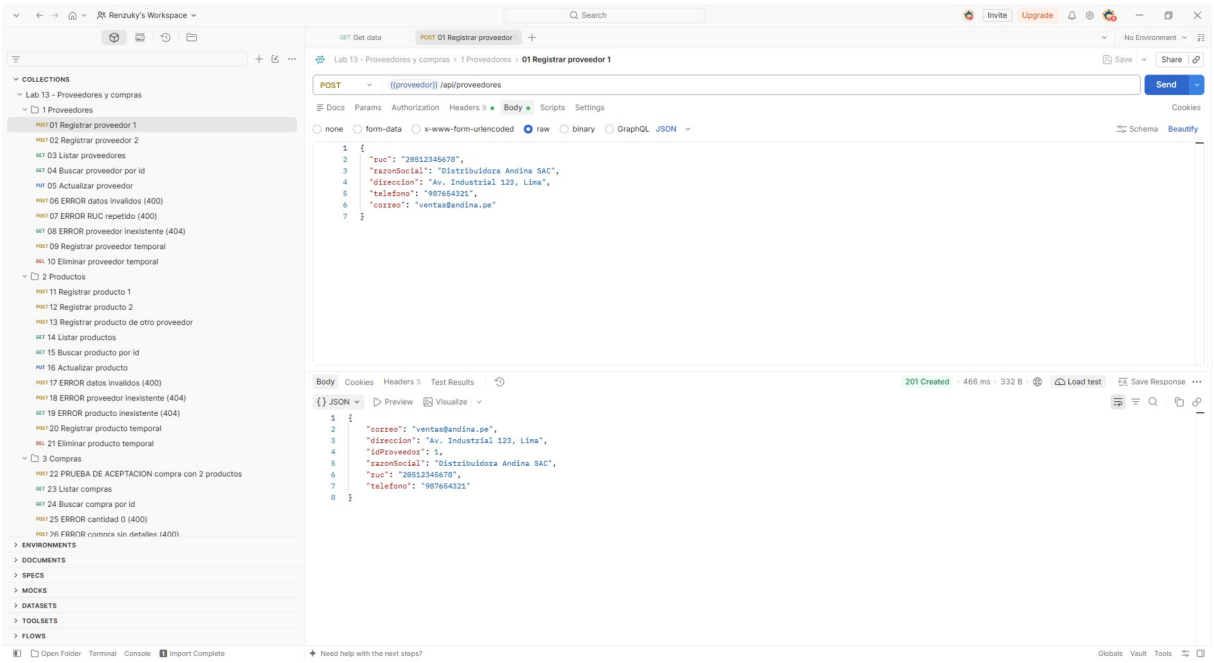



*Figura 1. Registro de un proveedor; responde 201 Created.*

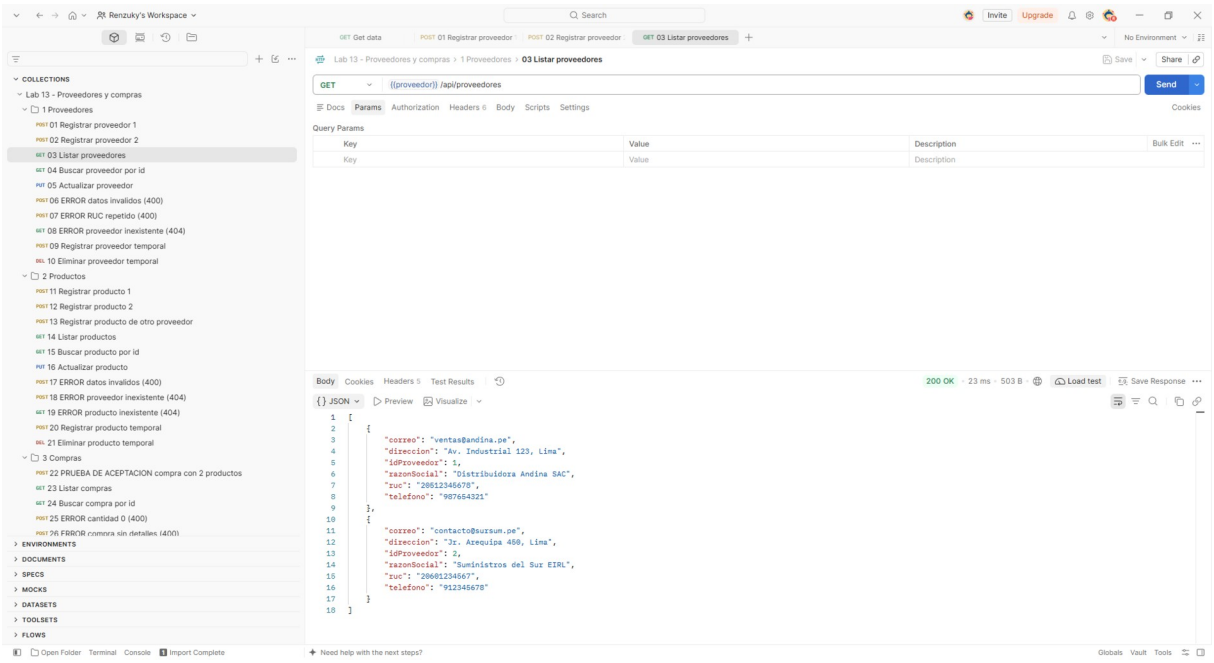



*Figura 2. Lista de proveedores registrados.*

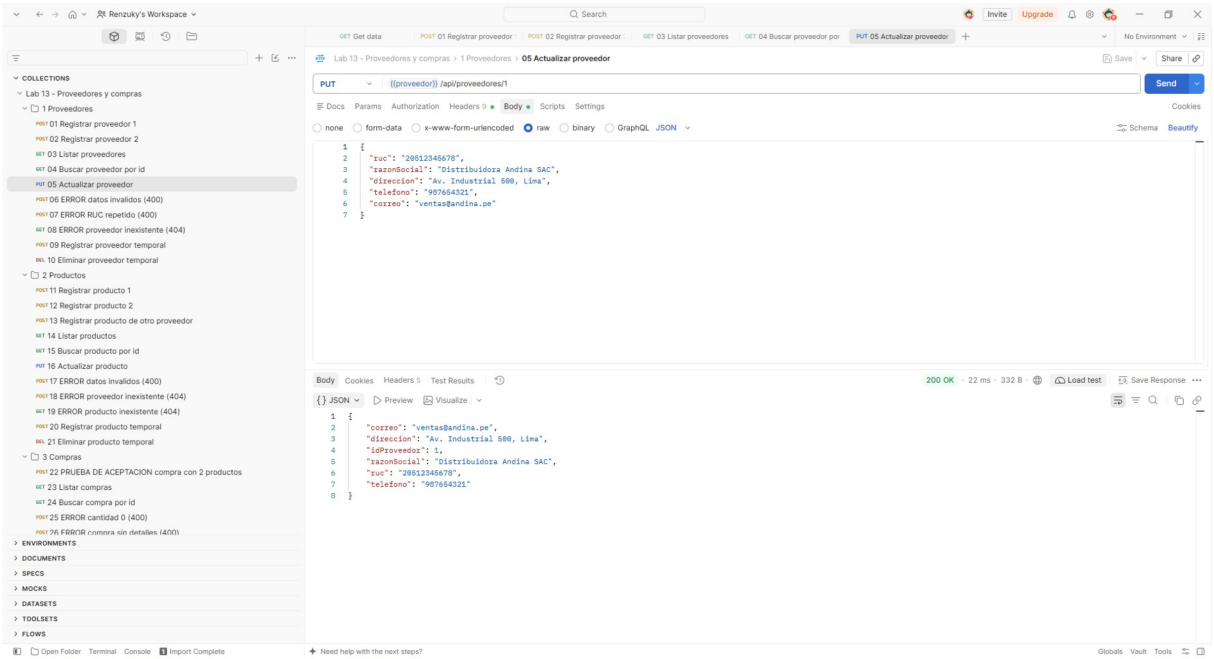



*Figura 3. Actualización de la dirección del proveedor.*

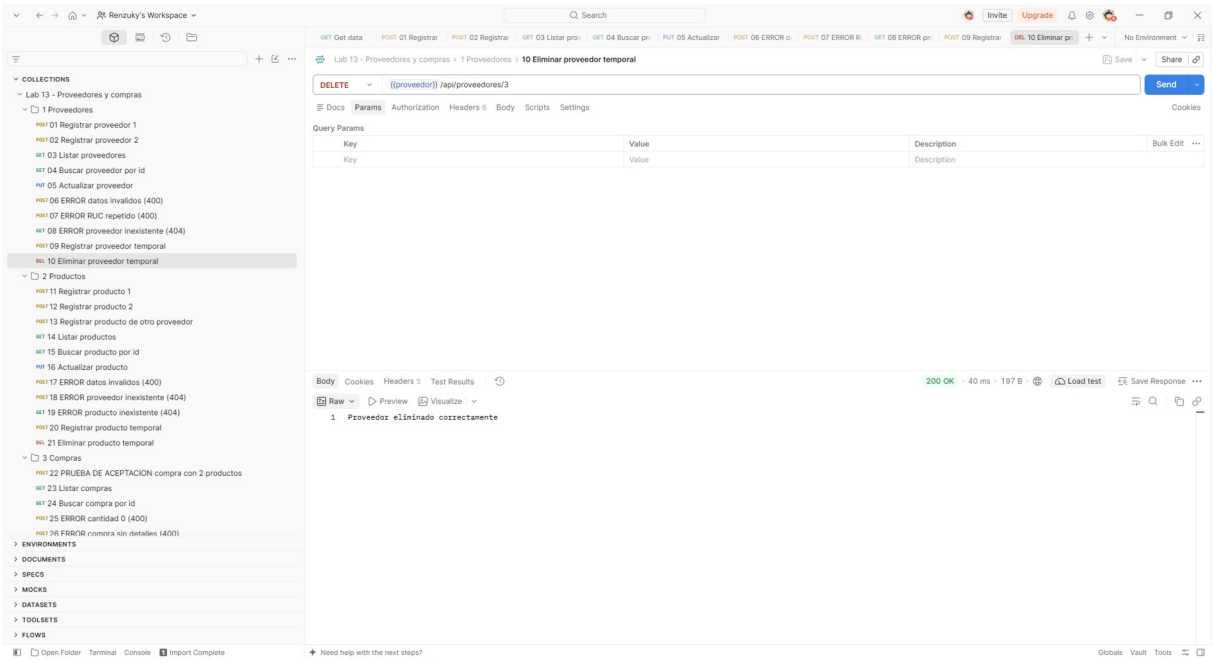



*Figura 4. Eliminación de un proveedor de prueba.*

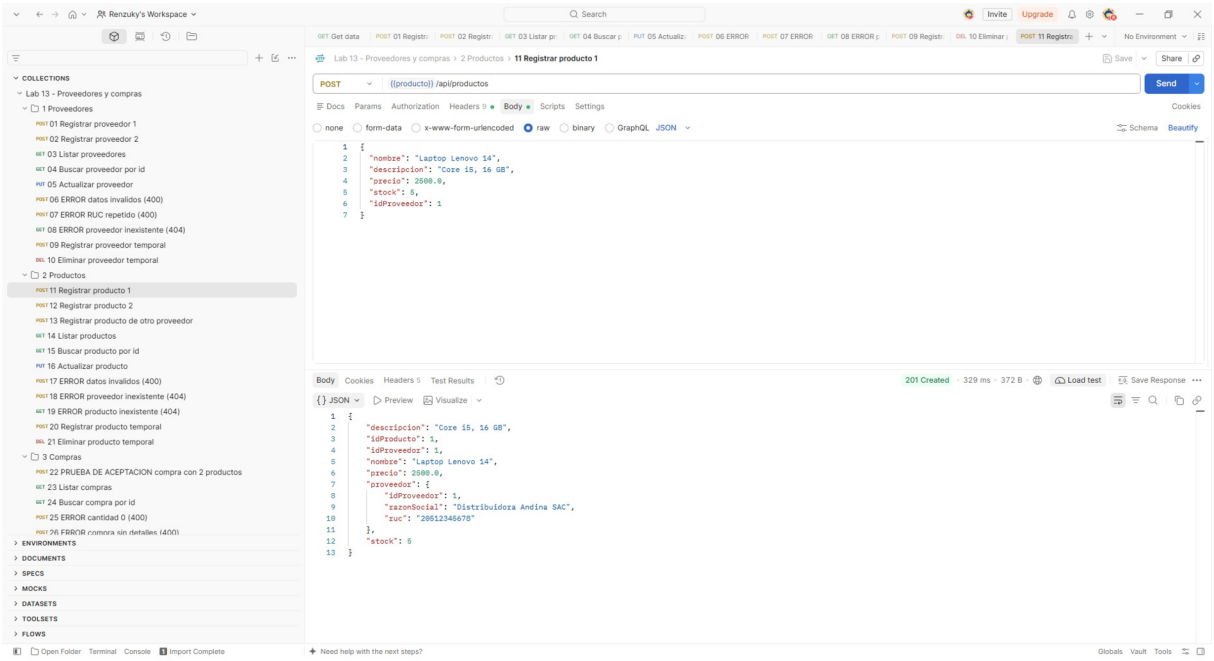



*Figura 5. Registro de un producto; la respuesta incluye los datos de su proveedor.*

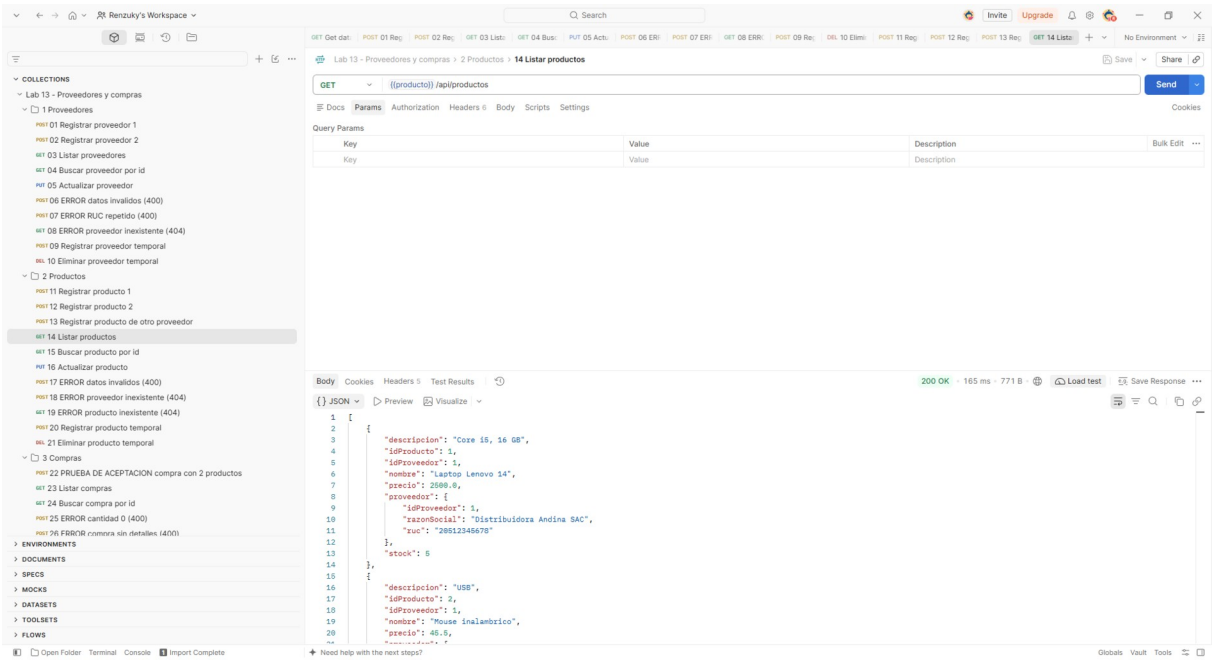



*Figura 6. Lista de productos registrados.*

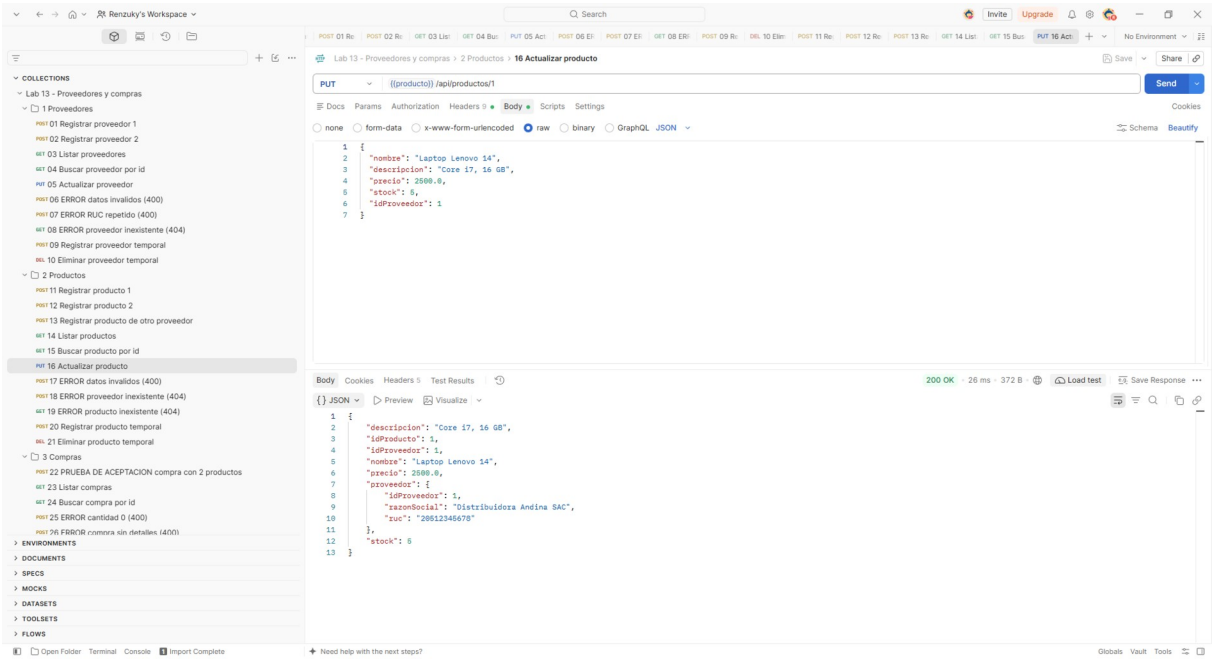



*Figura 7. Actualización de un producto.*

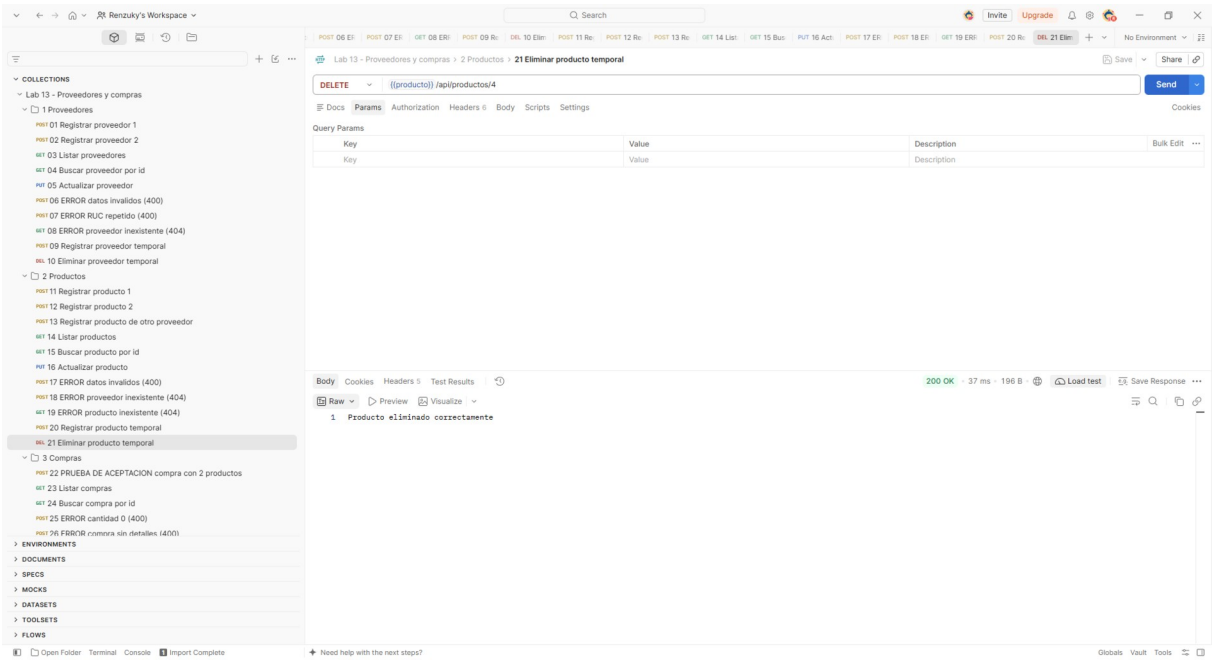



*Figura 8. Eliminación de un producto de prueba.*

## 2. Compra registra proveedor, fecha, total y detalles con producto, cantidad y precio.

La compra guarda el id del proveedor, la fecha y el total. Cada detalle guarda el id del producto, la cantidad, el precio y el subtotal. La compra y sus detalles están en la misma base, relacionados con una llave foránea, y se guardan juntos.

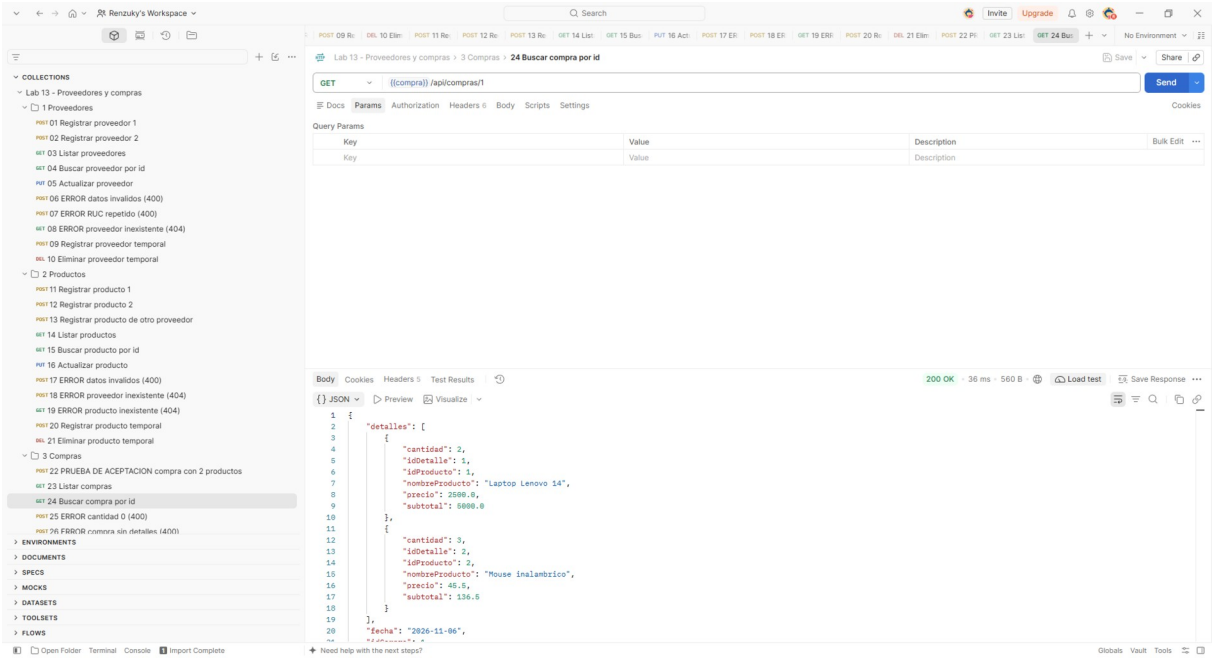



*Figura 9. Consulta de una compra con sus detalles.*

## 3. Validar RUC/documento, nombre y cantidades.

El RUC debe tener 11 dígitos, empezar con 10, 15, 17 o 20, y no repetirse. El nombre del producto y la razón social son obligatorios. La cantidad debe ser mayor a cero. Cuando un dato no cumple, la API responde 400 e indica qué campo falló.

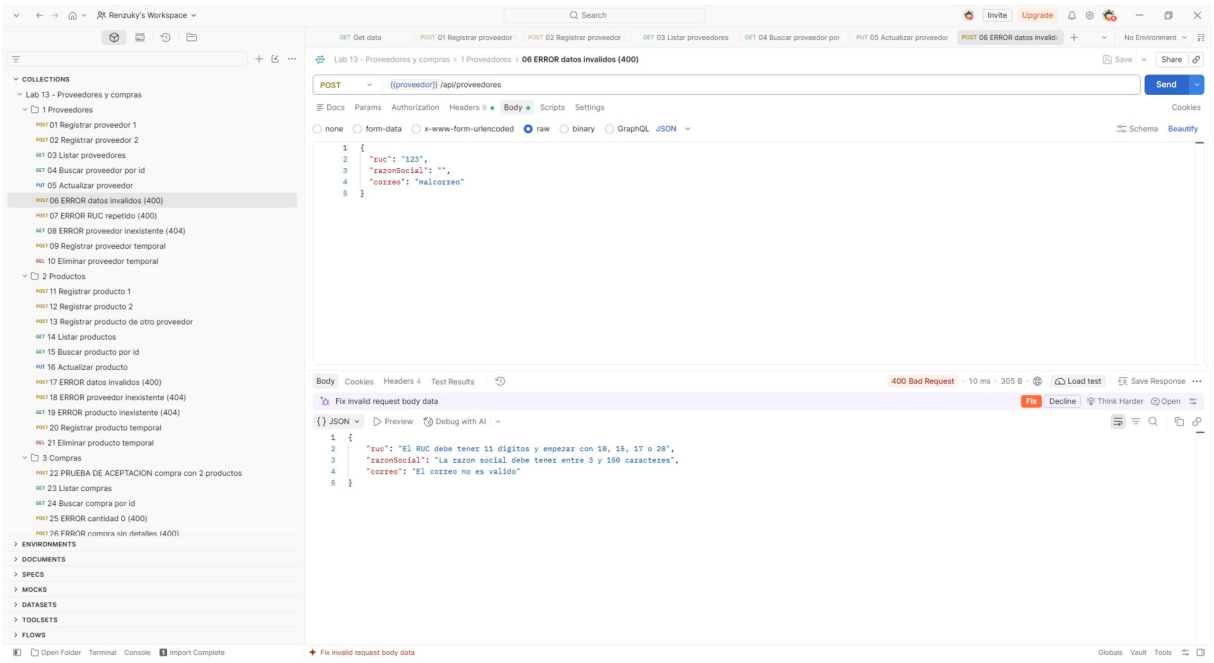



*Figura 10. RUC, razón social y correo inválidos.*

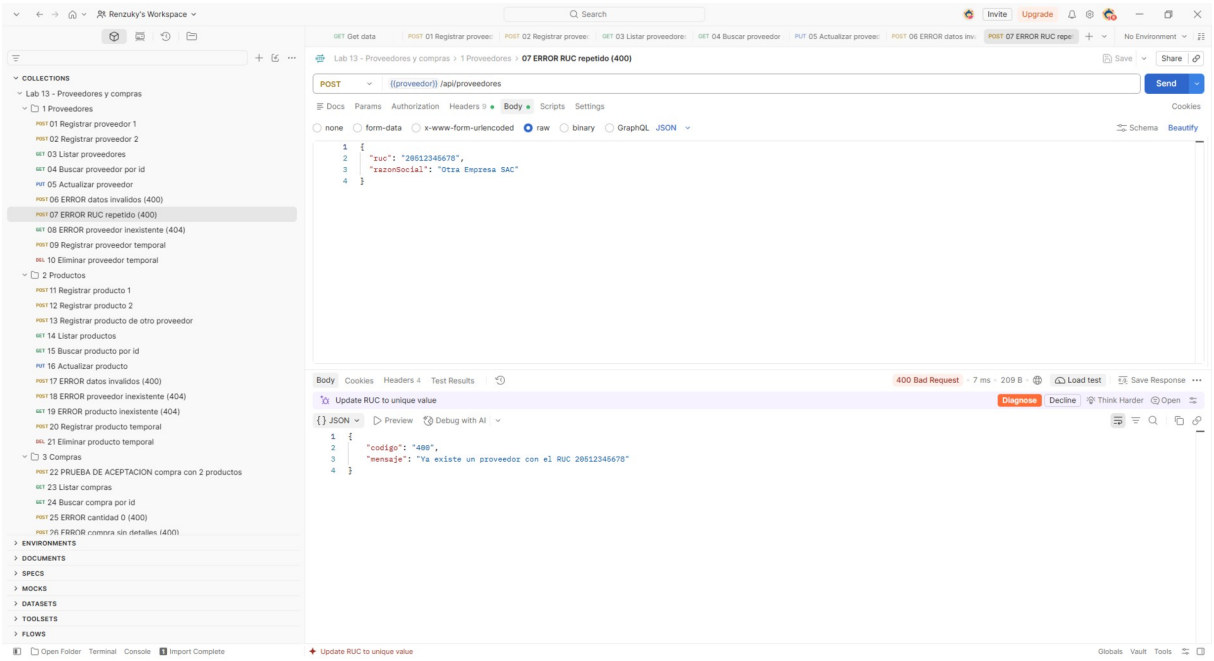



*Figura 11. RUC que ya estaba registrado.*

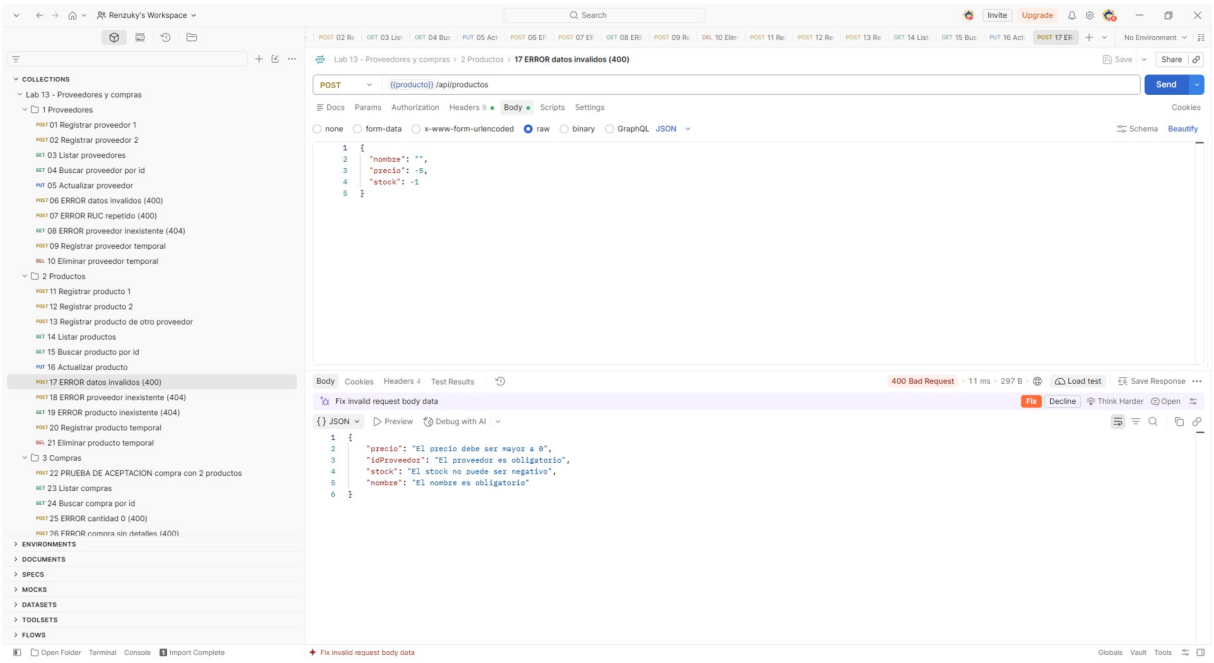



*Figura 12. Nombre vacío, y precio y stock negativos.*

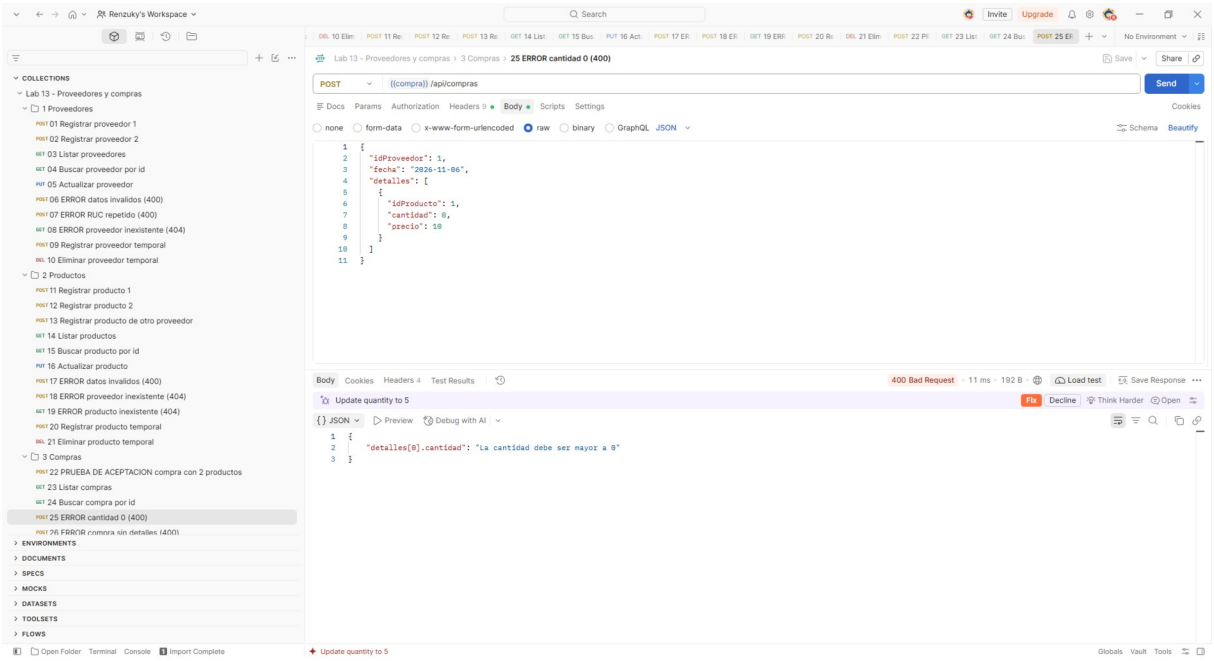



*Figura 13. Compra con cantidad cero.*

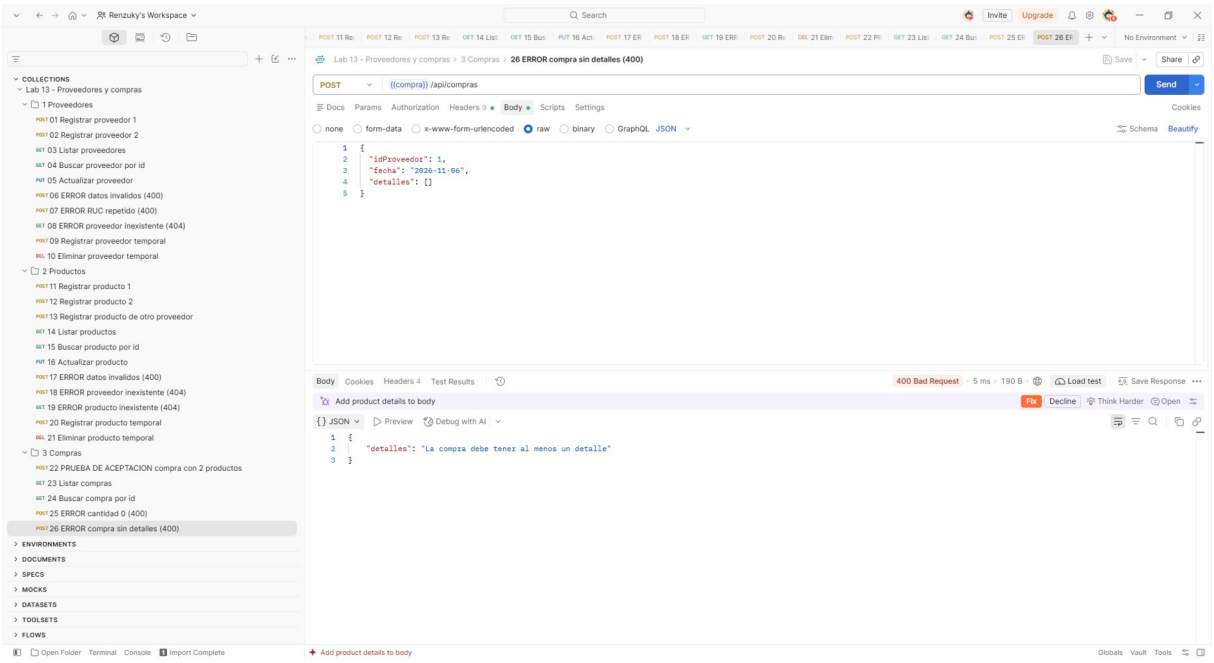



*Figura 14. Compra sin detalles.*

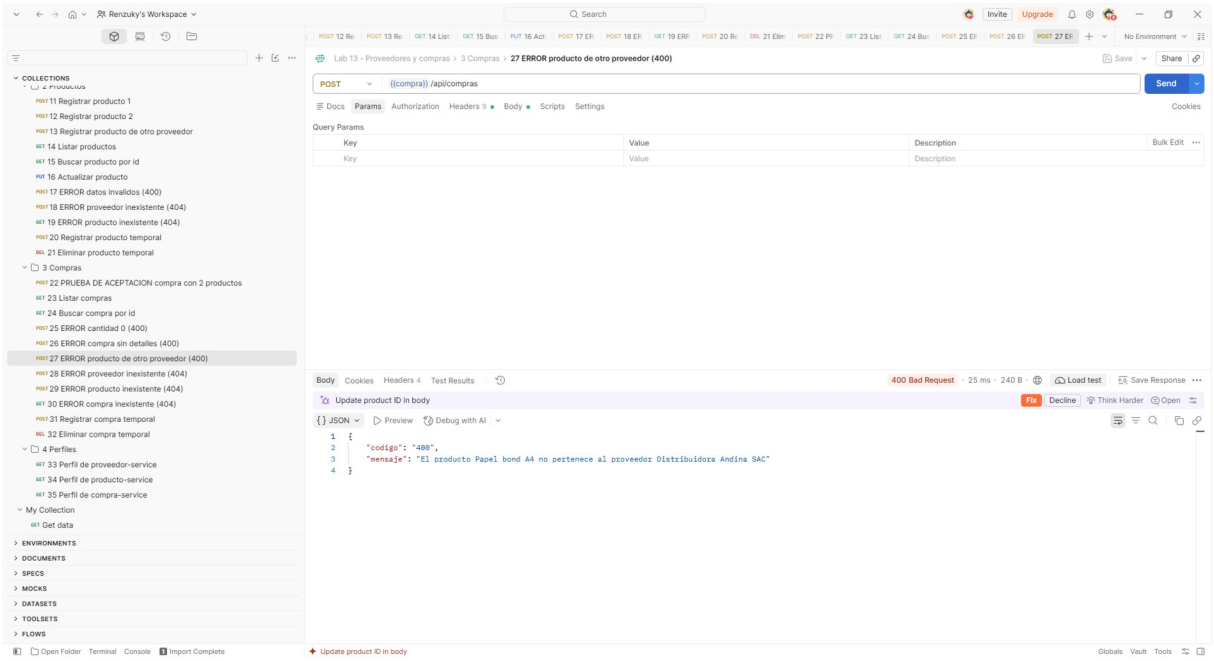



*Figura 15. Producto que pertenece a otro proveedor.*

## 4. Calcular subtotal por detalle y total de compra.

El servicio multiplica la cantidad por el precio para obtener cada subtotal, y suma los subtotales para obtener el total. El cliente no envía estos valores, así que no se pueden alterar desde afuera.

```
double total = 0;
for (DetalleCompraDTO item : dto.getDetalles()) {
    // Subtotal del detalle = cantidad x precio
    double subtotal = redondear(item.getCantidad() * item.getPrecio());
    total = total + subtotal;
}
// Total de la compra = suma de los subtotales
compra.setTotal(redondear(total));
```

**Bases de datos sugeridas:** bd_proveedor, bd_producto y bd_compra. En el perfil de desarrollo se llaman bd_proveedor_dev, bd_producto_dev y bd_compra_dev.

# Actividades de desarrollo

### 1. Crear los proyectos Maven necesarios y asignar un puerto distinto a cada servicio.

Se crearon cuatro proyectos Maven con Spring Initializr: el servidor Eureka y los tres microservicios. Se usó Spring Boot 4.1.2 y Java 25.

| Proyecto | Puerto | Responsabilidad |
|---|---|---|
| eureka-server | 8761 | Registro de los servicios |
| proveedor-service | 8081 | Administrar proveedores |
| producto-service | 8082 | Administrar productos |
| compra-service | 8083 | Registrar compras con sus detalles |

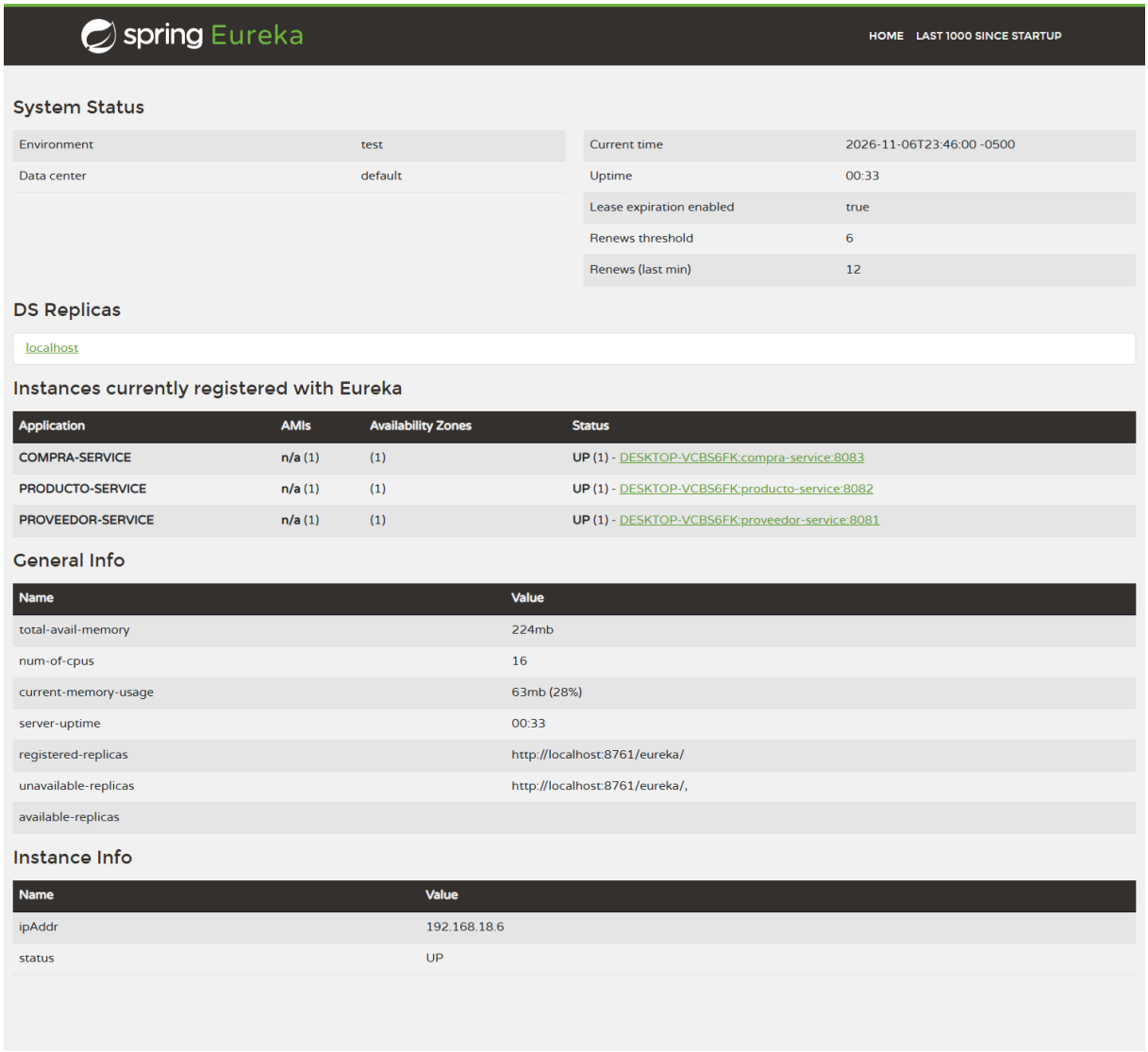



*Figura 16. Los tres microservicios registrados en Eureka.*

## 2. Configurar la conexión a MySQL y comprobar que cada servicio use su propia base de datos.

Cada servicio tiene su propia conexión y solo entra a su base. Por eso, cuando uno necesita un dato de otro, se lo pide por la API en vez de leer su tabla. En las imágenes se ven los datos guardados en cada base.

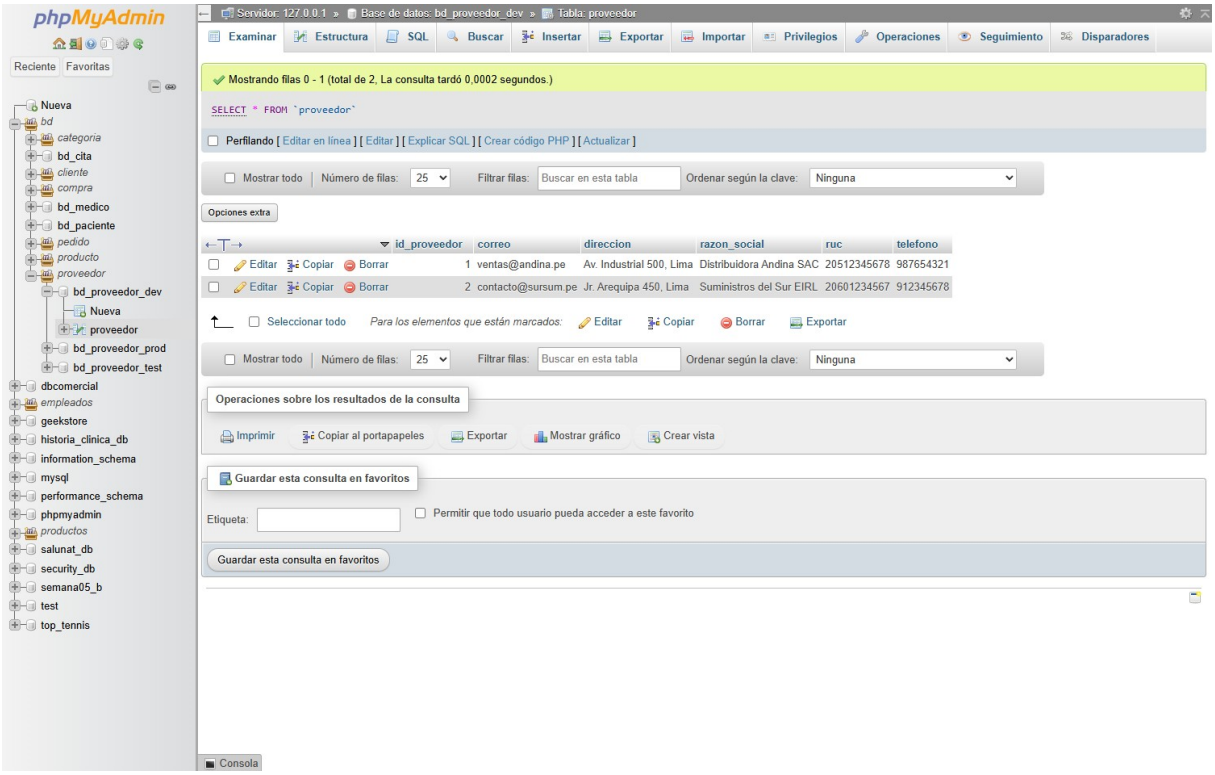



*Figura 17. Tabla de proveedores en su propia base.*

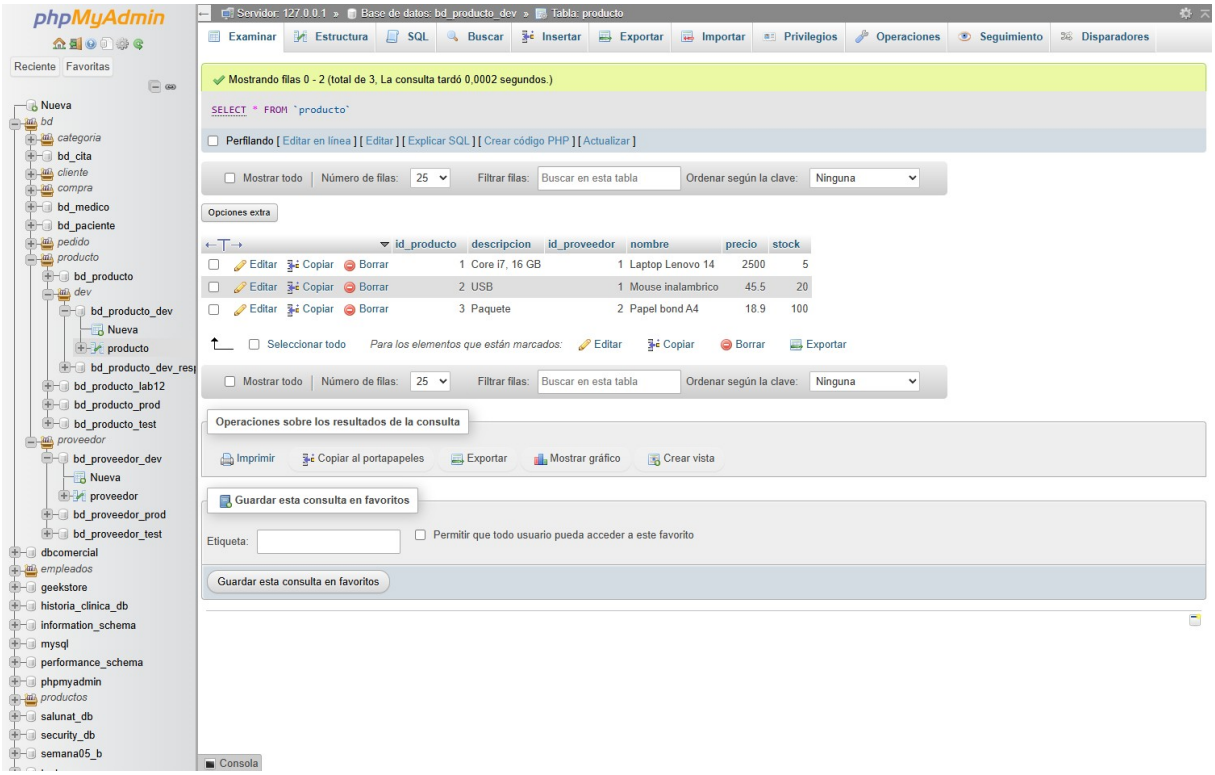



*Figura 18. Tabla de productos en su propia base.*

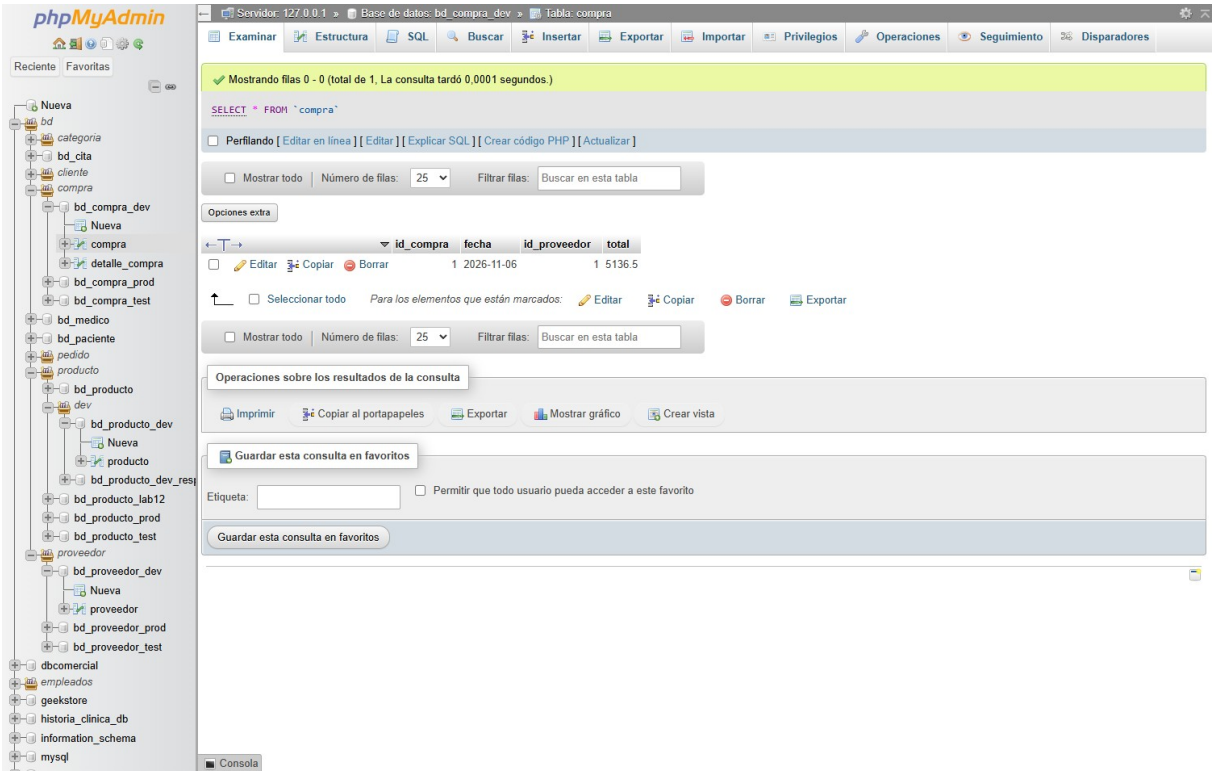



*Figura 19. Compra guardada con su total.*

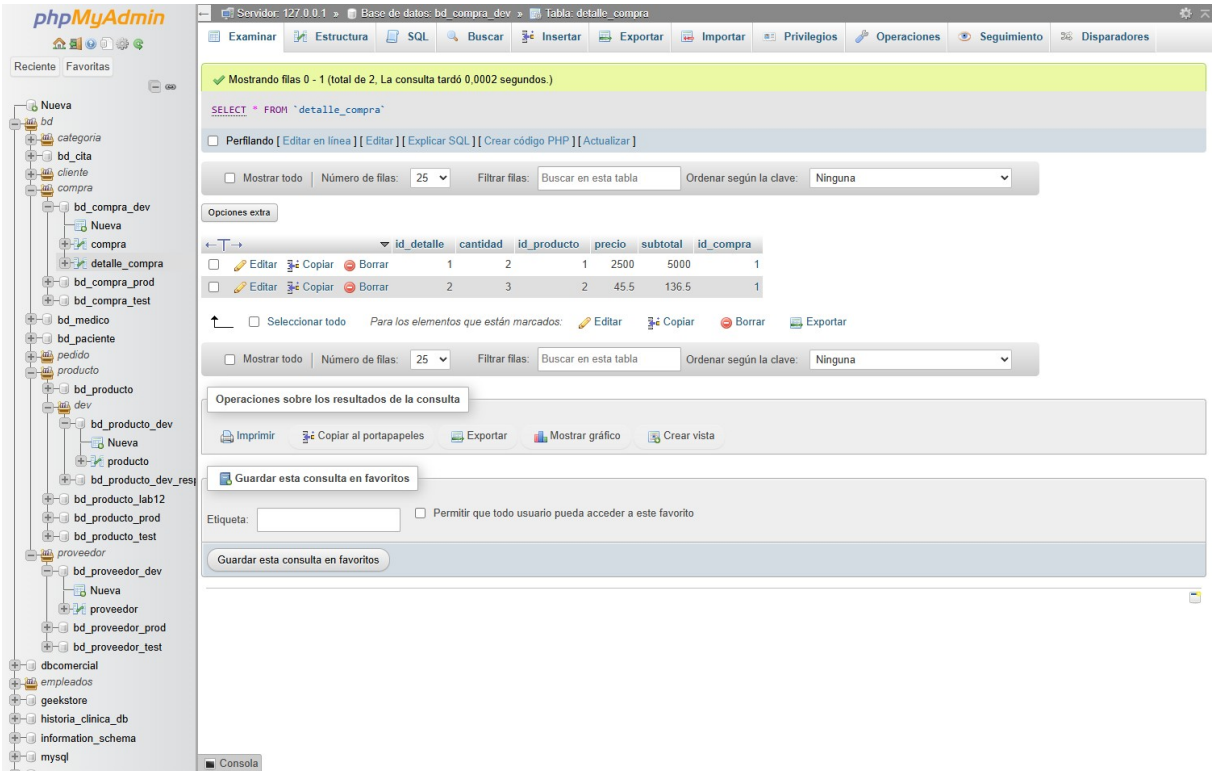



*Figura 20. Los dos detalles de la compra, con sus subtotales.*

## 3. Implementar entidades, repositorios, servicios y controladores REST.

Los tres microservicios tienen la misma organización: la entidad representa la tabla, el repositorio guarda y consulta, el servicio aplica las reglas y el controlador recibe las peticiones. También hay un paquete para los datos de entrada y otro para los errores. Los dos servicios que consultan a otros tienen además un paquete

cliente, donde se declara la consulta con Feign usando solo el nombre registrado en Eureka, sin dirección ni puerto.

```
@FeignClient(name = "producto-service")
public interface ProductoClient {

    @GetMapping("/api/productos/{id}")
    ProductoDTO obtenerProducto(@PathVariable("id") Long id);
}
```

**4. Agregar DTO y validaciones donde correspondan; manejar recursos inexistentes con respuestas HTTP apropiadas.**

Las peticiones llegan primero a un objeto de entrada con sus reglas de validación, y solo si son correctas pasan a la entidad. Un manejador general convierte cada problema en una respuesta clara: 400 para datos inválidos, 404 si el registro no existe y 503 si otro microservicio está apagado.

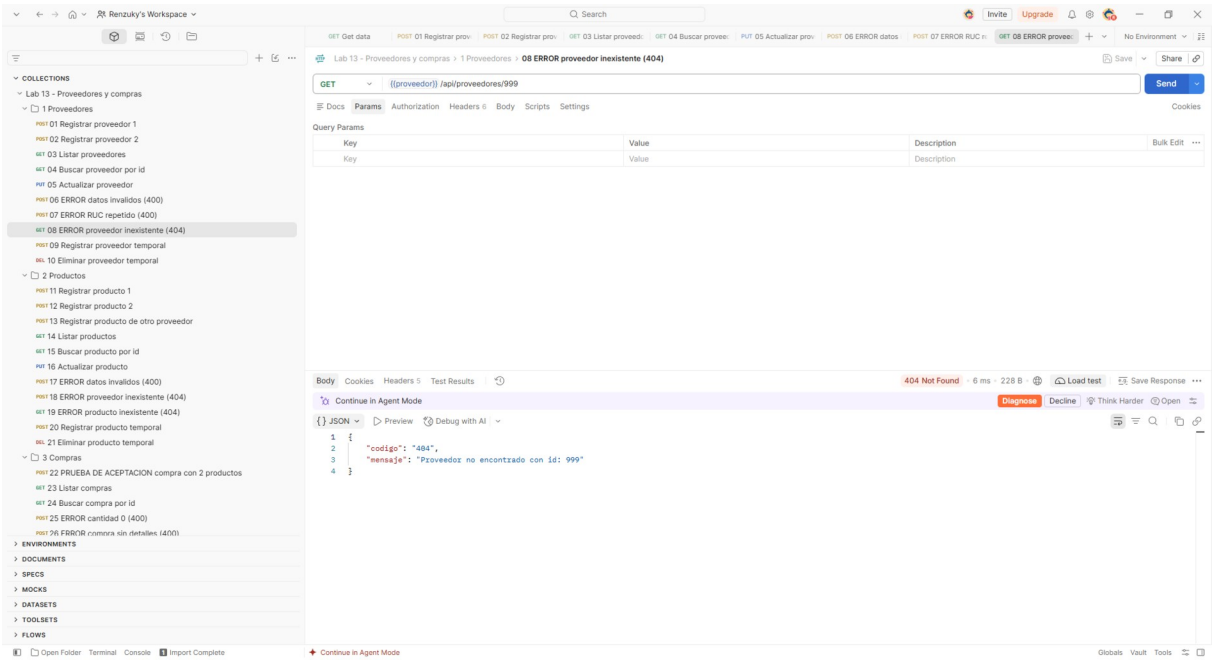



*Figura 21. Proveedor inexistente.*

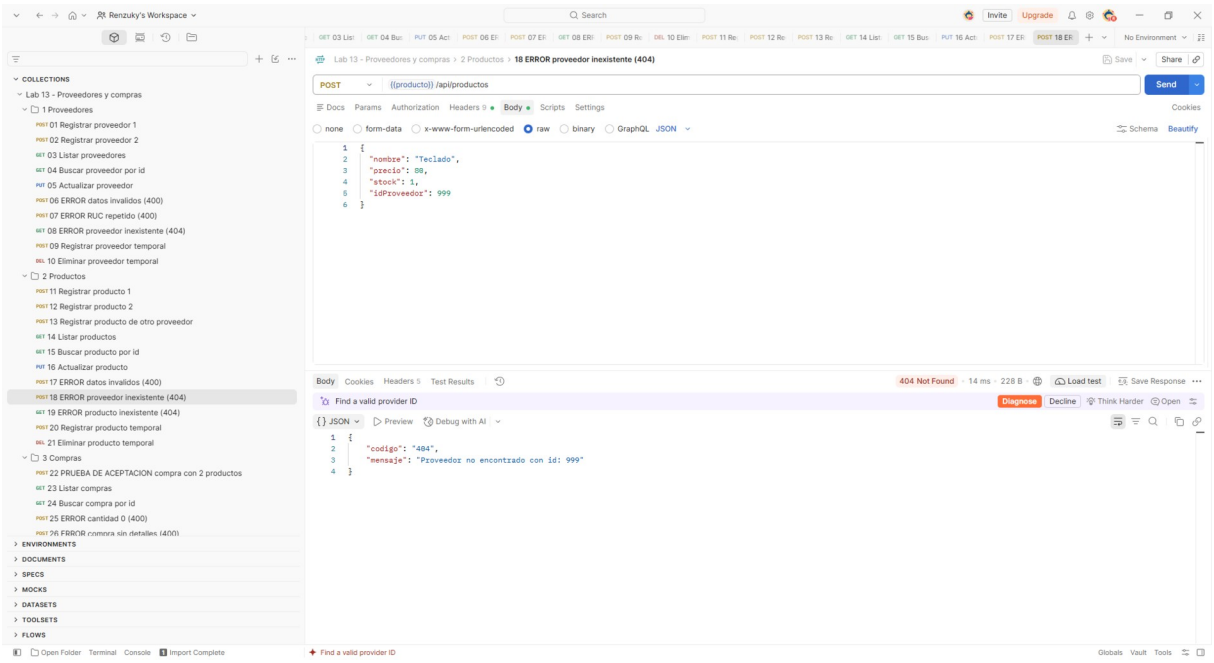



*Figura 22. Producto con un proveedor que no existe.*

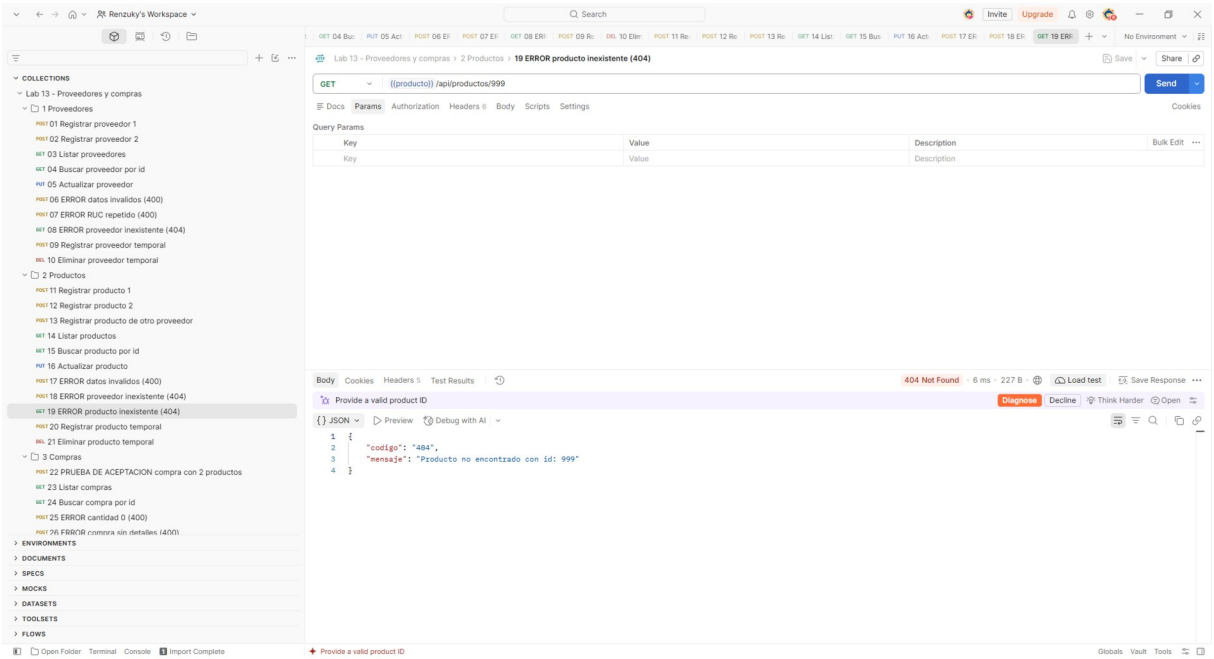



*Figura 23. Producto inexistente.*

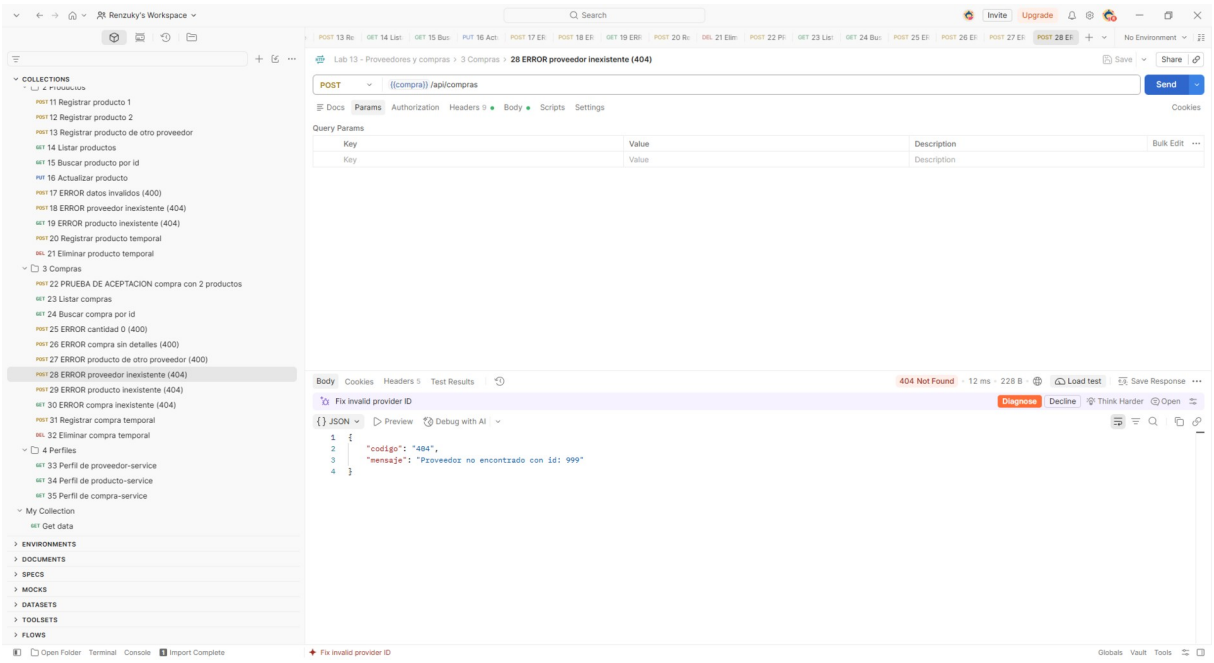



*Figura 24. Compra con un proveedor que no existe.*

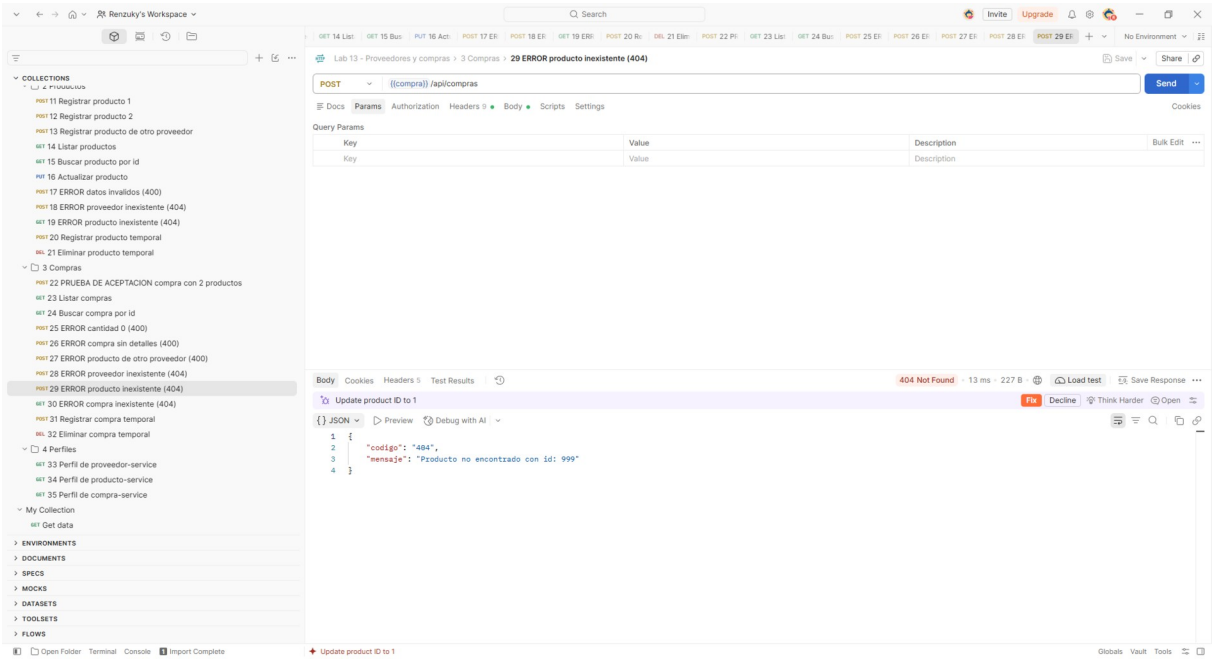



*Figura 25. Compra con un producto que no existe.*

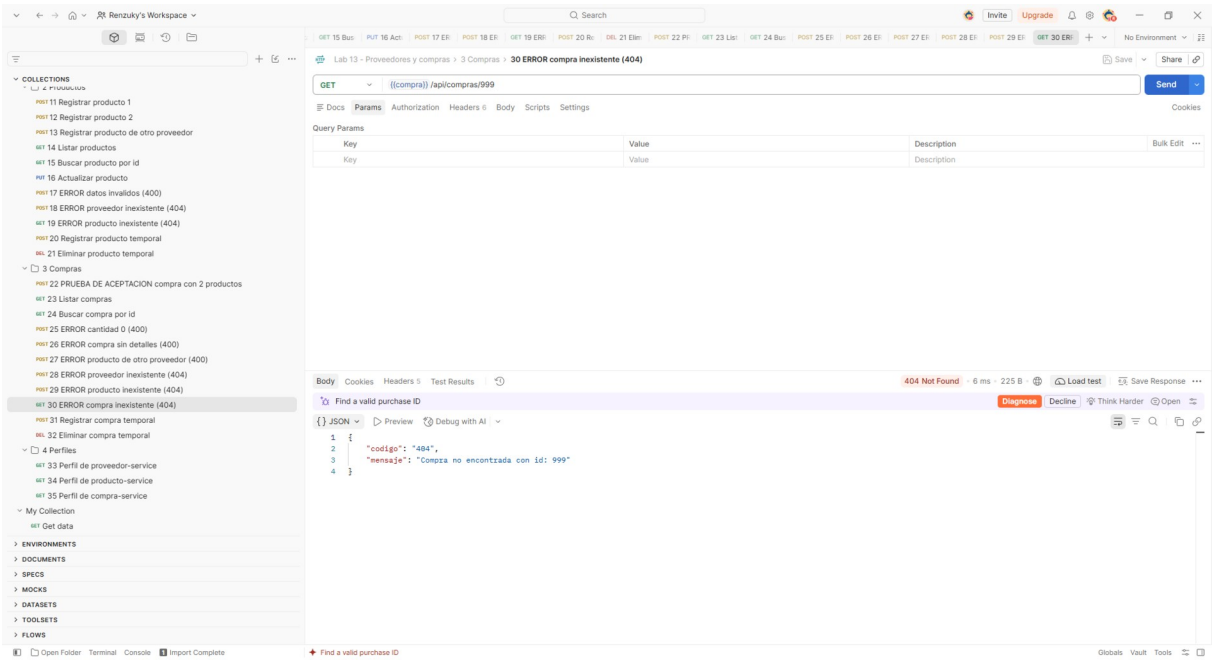



*Figura 26. Compra inexistente.*

## 5. Configurar application.yml y perfiles dev/prod; documentar qué valores cambian entre ambientes.

El perfil se elige con una variable de entorno y, si no se indica, se usa desarrollo. Además de los perfiles dev y prod se agregó uno de pruebas, para no mezclar datos entre ambientes. Esto es lo que cambia en cada uno:

| Valor | dev | test | prod |
|---|---|---|---|
| Puerto | 8081 a 8083 | 8181 a 8183 | 9081 a 9083 |
| Base de datos | bd_*_dev | bd_*_test | bd_*_prod, por variable de entorno |
| Usuario y contraseña | Fijos | Fijos | Por variables de entorno |
| Tablas | Se crean o actualizan | Se crean o actualizan | Solo se verifican; se crean con el script |
| Mostrar SQL | Sí | Sí | No |

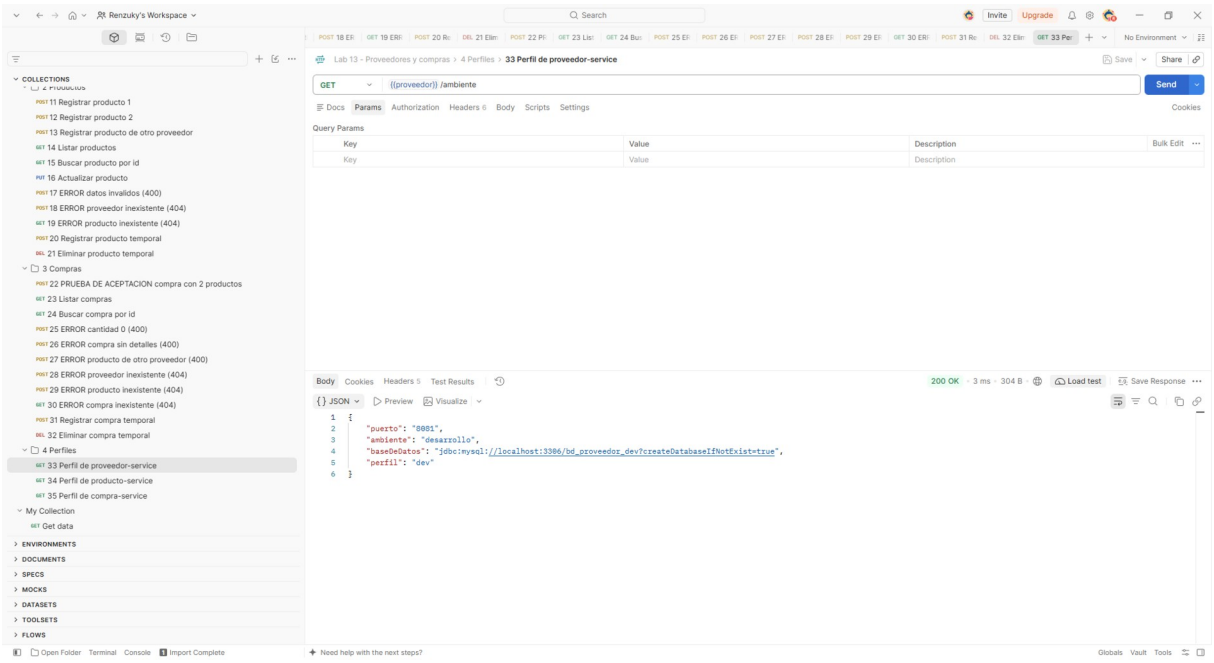



*Figura 27. proveedor-service corriendo con el perfil dev.*

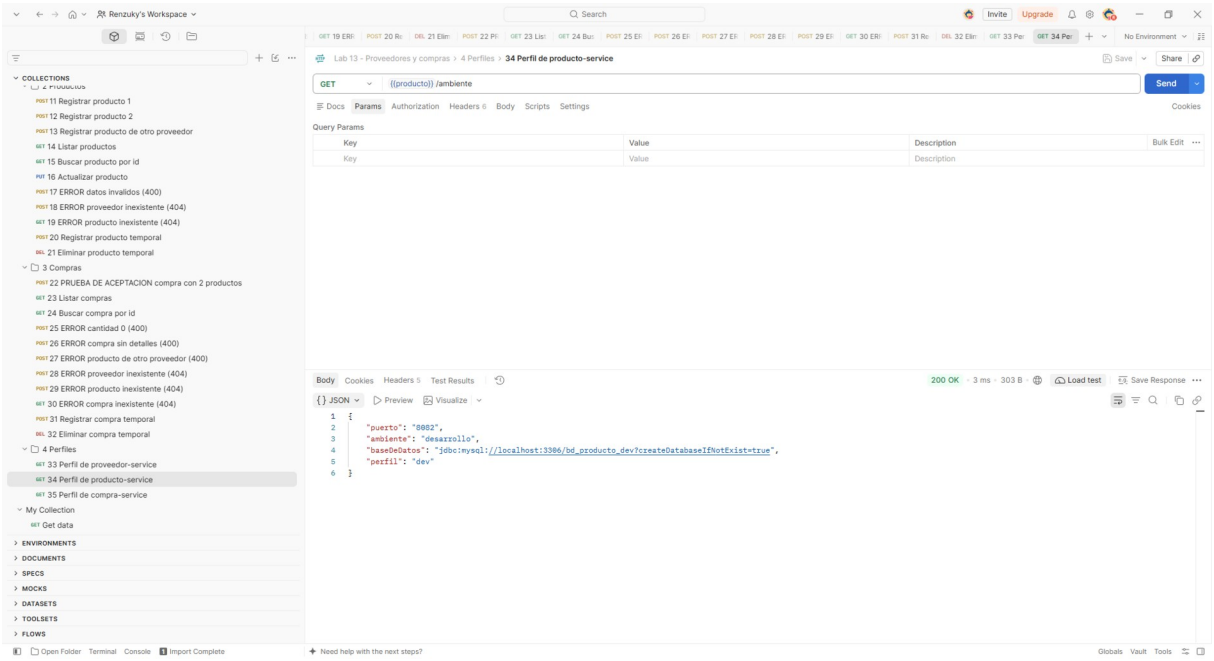



*Figura 28. producto-service corriendo con el perfil dev.*

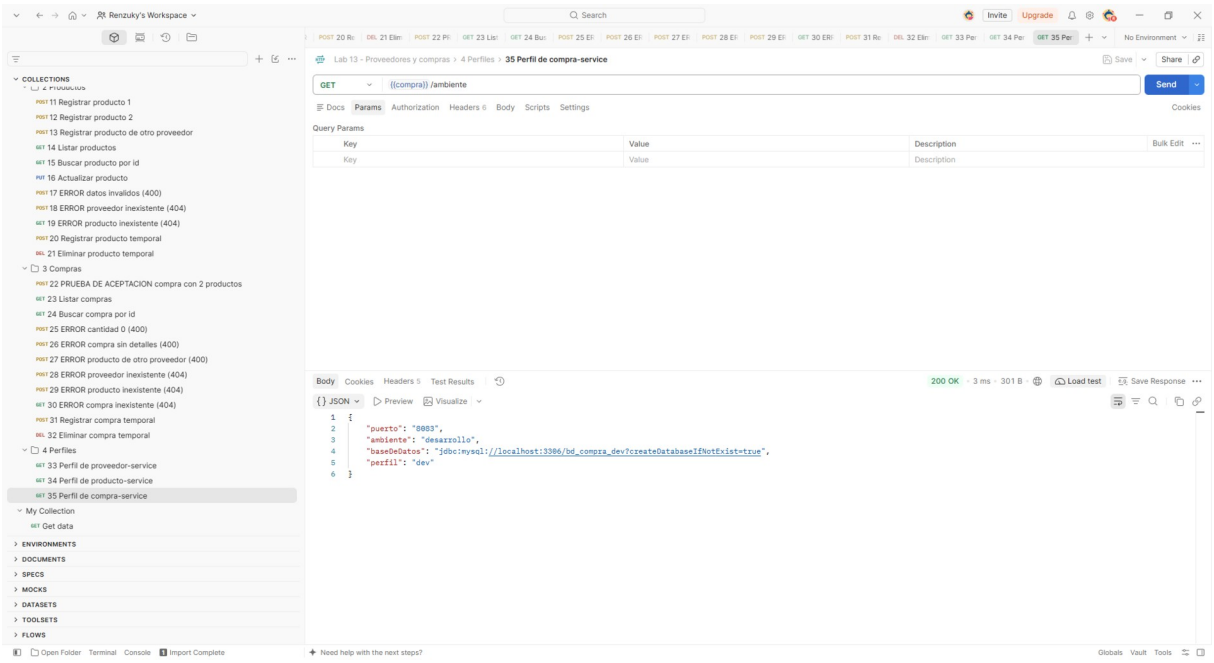



*Figura 29. compra-service corriendo con el perfil dev.*

**6. Probar las operaciones con Postman y registrar evidencias.**

Se creó una colección con 35 peticiones, entre casos exitosos y de error, y se ejecutó en Postman contra los servicios encendidos. Todas devolvieron el código esperado. Las imágenes de este informe son esas respuestas, y la colección está en la carpeta postman del repositorio.

# Prueba de aceptación específica

**Registrar una compra con dos productos y presentar el cálculo del total.** Se registró una compra con dos productos. El cálculo fue el siguiente:

| Producto | Cantidad | Precio | Subtotal |
|---|---|---|---|
| Laptop Lenovo 14 | 2 | 2500.00 | 5000.00 |
| Mouse inalámbrico | 3 | 45.50 | 136.50 |
| Total de la compra | | | 5136.50 |

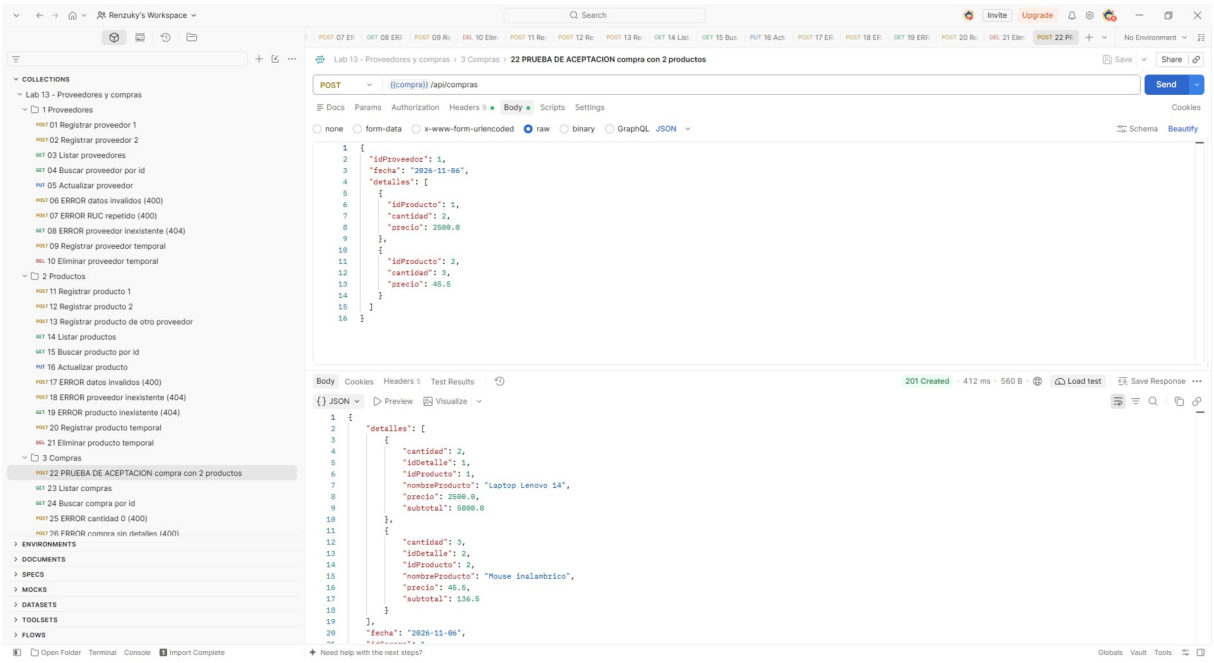



*Figura 30. Registro de la compra; responde 201 Created con los subtotales.*

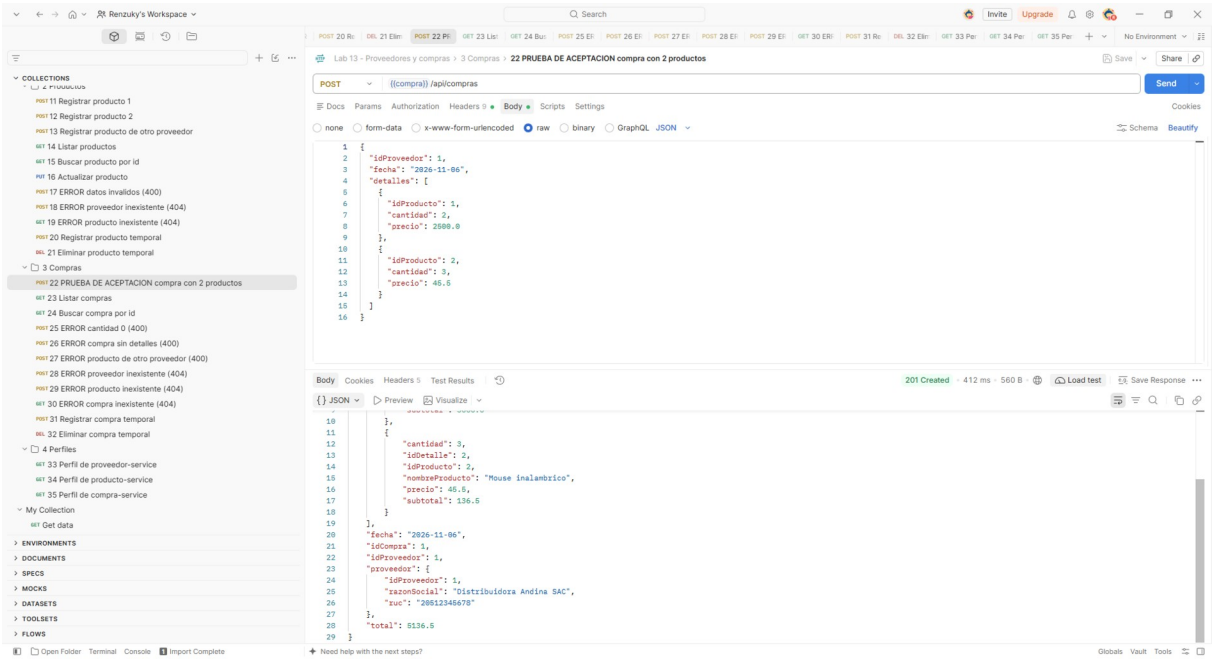



*Figura 31. Final de la misma respuesta, con el total de 5136.5.*

# Evidencias que debe entregar

### Código fuente y README de ejecución.

El código, el README con los pasos de ejecución, el script SQL y la colección de Postman están en el repositorio: https://github.com/RXDG2908/eval03-proveedores-compras

## Diagrama de microservicios y bases de datos.

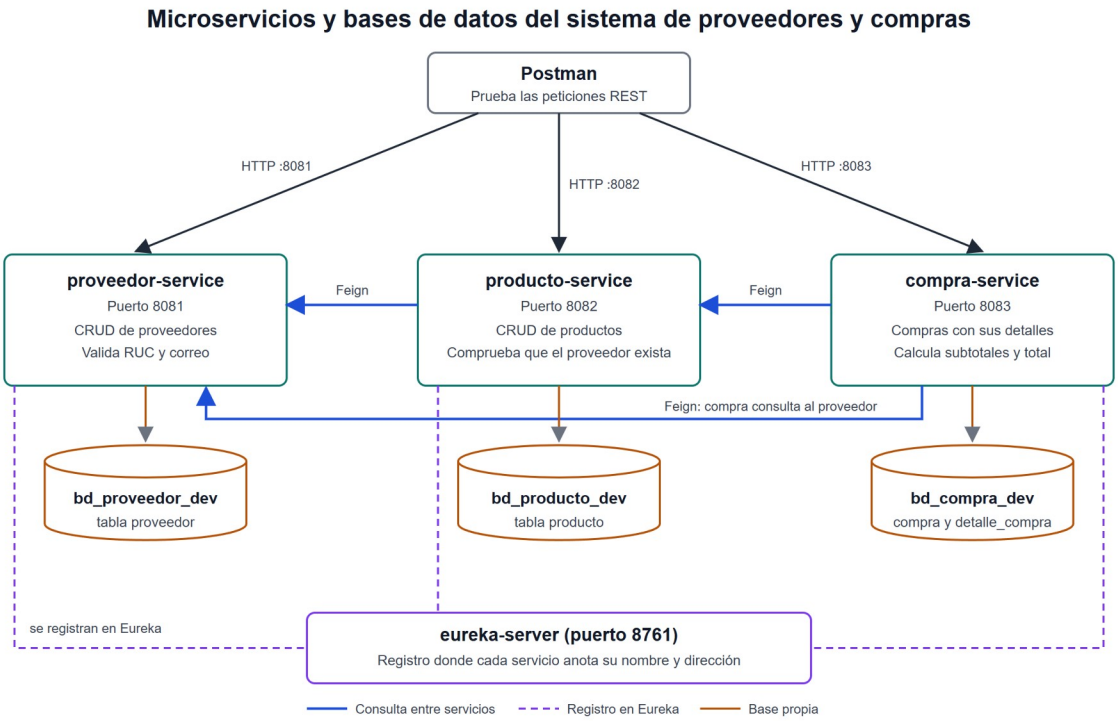



*Figura 32. Microservicios, responsabilidades y bases de datos.*

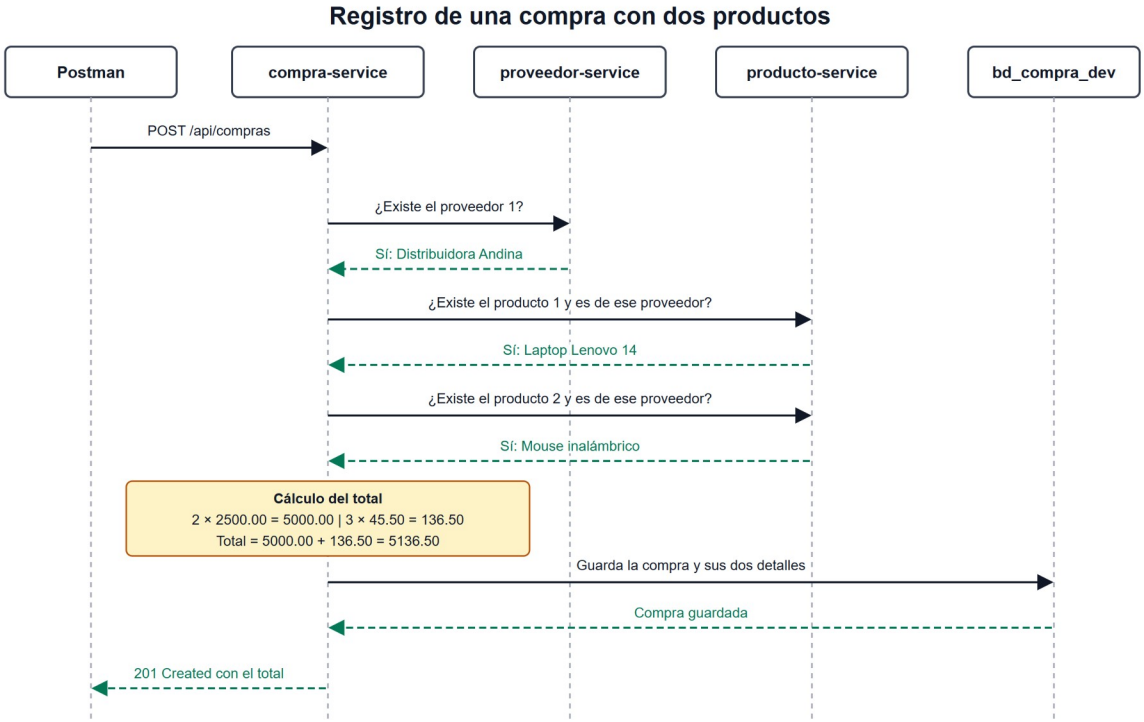



*Figura 33. Pasos para registrar una compra.*

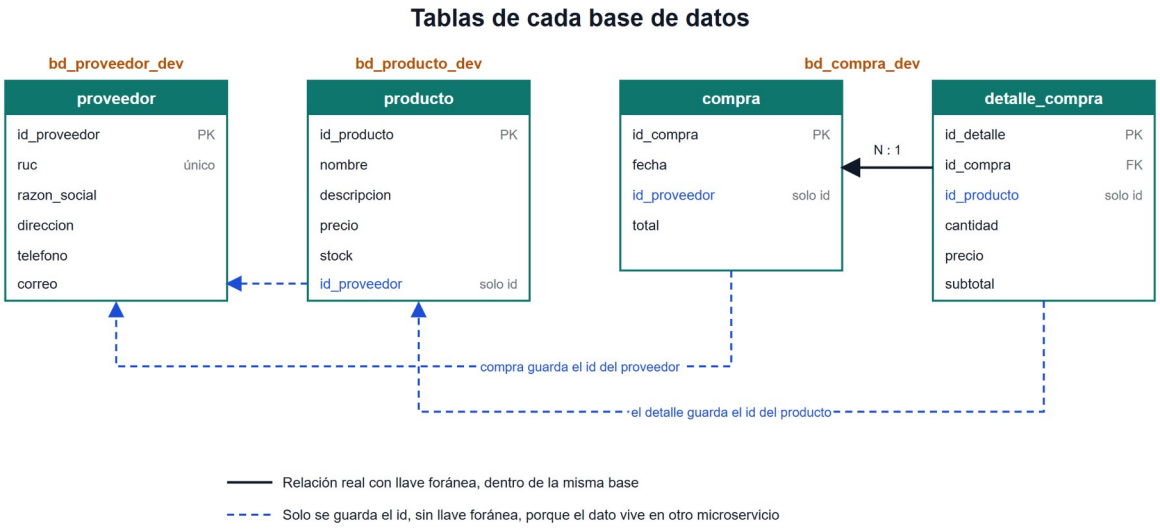



*Figura 34. Tablas de cada base de datos.*

**Capturas o colección de Postman: al menos un caso exitoso y un caso de error.**

Las capturas están en los requerimientos y actividades anteriores. Por ejemplo, la Figura 1 muestra un caso exitoso y la Figura 10 un caso de error.

**Cinco conclusiones.**

**1.** Separar el sistema en microservicios ayuda a que cada parte tenga una sola responsabilidad y su propia base de datos, de modo que un cambio en una no obliga a tocar las demás.

**2.** Eureka evita escribir direcciones y puertos fijos, porque cada servicio se registra con su nombre y los demás lo encuentran por ese nombre.

**3.** Con Feign, consultar a otro servicio se declara con una interfaz sencilla, parecida a escribir un controlador, y el código queda fácil de leer.

**4.** Validar los datos antes de procesarlos, junto con un manejador general de errores, hace que la API responda con mensajes claros y con el código HTTP correcto.

**5.** Los perfiles permiten usar la misma aplicación en distintos ambientes, cambiando solo la configuración y sin tocar el código.

**Dificultad encontrada y solución.**

Al probar el servicio de productos, todas las peticiones respondían error 500 aunque el código estaba bien. La causa era que ya existía una base de datos con el mismo nombre, creada en una práctica anterior, cuya tabla tenía otras columnas. Se movió esa tabla a una base de respaldo para no perder sus datos y el servicio creó su tabla correctamente. Desde entonces revisé que cada proyecto use un nombre de base propio antes de ejecutarlo.

# Pregunta para la sustentación

**Pregunta.** Explique por qué separó el sistema en estos servicios, cómo persiste los datos y qué ocurre cuando la API recibe información inválida.

**Respuesta.** Separé el sistema en tres servicios porque proveedores, productos y compras son responsabilidades distintas: cada uno puede cambiar y ejecutarse por su cuenta. Los datos se guardan con

Spring Data JPA en MySQL, cada servicio en su propia base. Como no comparten tablas, una compra guarda solo el id del proveedor y de cada producto, y consulta sus datos por la API con Feign; solo la compra y sus detalles tienen una relación directa, porque viven en la misma base. Cuando la API recibe información inválida, la validación la detiene antes de guardarla y responde 400 con el campo y el motivo. Si el registro no existe responde 404, y en ninguno de esos casos se guarda nada.